

Spoiler Detection

Simplified BERT Pipeline

Technical Documentation

NLP Binary Text Classification

Model: BERT-base-uncased + MLP Classifier

Dataset: Goodreads Reviews (Spoiler) — 200,000 records

<i>Nouran Hassan</i>	<i>2022/00062</i>
<i>Roaa Khaled</i>	<i>2022/05885</i>
<i>Yahia Tamer</i>	<i>2022/02264</i>
<i>Malak Mohamed</i>	<i>2022/07005</i>
<i>Daniel Michel</i>	<i>2022/02556</i>

1. Project Overview

This project implements a sentence-level spoiler detection system for reviews using a fine-tuned BERT model. The goal is to automatically identify sentences in Goodreads reviews that contain plot spoilers, enabling users to read reviews safely without having story details revealed.

Problem Statement

Online reviews frequently contain plot spoilers mixed with general commentary. Manually tagging spoiler sentences is impractical at scale. This pipeline builds a binary classifier that labels each sentence as either a spoiler or non-spoiler at inference time.

1.1 Key Design Decisions

- Sentence-level granularity — operates on individual sentences rather than full reviews, allowing fine-grained spoiler tagging.
- Frozen BERT encoder — BERT weights are frozen during training to reduce computation while still leveraging pre-trained contextual representations.
- Auxiliary features — relative sentence position and log-length are concatenated to the BERT CLS embedding to provide structural signals.
- Focal loss — used to address the class imbalance between spoiler and non-spoiler sentences.

2. Dataset

2.1 Source

The dataset is the Goodreads Reviews (Spoiler) dataset stored as a JSONL file (`goodreads_reviews_spoiler.json`). Each line is a JSON object representing one book review, where the `review_sentences` field contains a list of (`is_spoiler`, `sentence`) pairs.

2.2 Loading & Structure

The first 200,000 reviews are loaded into memory:

```
with open('goodreads_reviews_spoiler.json', 'r') as f:
    for i, line in enumerate(f):
        if i >= 200_000: break
        records.append(json.loads(line))
```

2.3 Schema

Field	Type	Description
<code>review_sentences</code>	List[Tuple]	List of (<code>is_spoiler</code> : bool, <code>sentence</code> : str) pairs

Field	Type	Description
is_spoiler	int (0/1)	Binary label — 1 if the sentence is a spoiler
sentence	str	Raw sentence text from the review
_rel_pos	float	Relative position of sentence in its review (pos / total)
_len_log	float	Natural log of sentence character length

2.4 Data Statistics

After expanding from review-level to sentence-level, the raw dataset yields millions of sentences. The pipeline samples 199,000 sentences for tractable training:

Split	Size	Ratio
Training	139,300	70%
Validation	29,850	15%
Test	29,850	15%

3. Text Preprocessing

3.1 Pipeline Steps

Each raw sentence passes through the following preprocessing steps before being fed to the BERT tokenizer:

Step	Operation	Rationale
1. Lowercase	<code>text.lower()</code>	Normalize case variation
2. Strip non-alpha	<code>re.sub(r'^[a-z\s]', "", text)</code>	Remove punctuation and digits
3. Tokenize	<code>nltk.word_tokenize(text)</code>	Language-aware tokenization
4. Remove stopwords	NLTK English stopwords	Reduce noise from common words
5. Lemmatize	WordNetLemmatizer	Reduce words to base forms

3.2 Cleaning & Deduplication

- Empty sentences — dropped after preprocessing (`sentence_clean == ""`).
- Ambiguous labels — sentences that appear as both spoiler and non-spoiler across different reviews are removed to prevent label noise.
- Final sampling — 199,000 sentences sampled with `random_state=42` for reproducibility.

Note on Preprocessing vs. BERT

The custom preprocessing (stopword removal, lemmatization) is applied before BERT tokenization on the cleaned text. This is a deliberate simplification — standard BERT pipelines typically pass raw text to the tokenizer. The cleaned text reduces vocabulary noise at the cost of some semantic information.

4. Model Architecture

4.1 BertSpoilerClassifier

The model is a two-stage neural network combining a frozen BERT encoder with a lightweight MLP classifier head:

Component	Details
Base encoder	bert-base-uncased (12 layers, 768 hidden dim, 110M params)
Encoder mode	Frozen (torch.no_grad()) — no fine-tuning
Input to MLP	CLS token embedding [B, 768] + rel_pos [B,1] + len_log [B,1] = [B, 770]
MLP Layer 1	Dropout(0.1) → Linear(770, 256) → ReLU
MLP Layer 2	Dropout(0.1) → Linear(256, 1)
Output	Raw logit → sigmoid → binary probability
Loss function	Focal Loss (alpha=0.25, gamma=2.0)

4.2 Focal Loss

Standard cross-entropy loss performs poorly when classes are imbalanced (far fewer spoiler sentences than non-spoiler). Focal Loss down-weights easy, well-classified examples and focuses learning on hard, misclassified ones:

```
loss = -(
    alpha * y * (1 - p)^gamma * log(p)      # spoiler term
    + (1-alpha) * (1-y) * p^gamma * log(1-p) # non-spoiler term
)
```

Hyperparameter	Value	Effect
alpha	0.25	Weights the spoiler class (minority) more heavily
gamma	2.0	Reduces loss for easy examples; focuses on hard cases
eps	1e-6	Numerical stability clamp for log(0) prevention

4.3 Auxiliary Features

Two hand-crafted features supplement the BERT CLS embedding:

- Relative position (`_rel_pos`) — the sentence's ordinal position divided by total review length. Spoilers are hypothesized to appear more frequently near the end of reviews.
- Log sentence length (`_len_log`) — log of character count. Longer, more descriptive sentences may correlate with plot details.

5. Training Configuration

Parameter	Value
Epochs	3
Batch size	16
Max token length	128
Optimizer	AdamW (lr=2e-5, eps=1e-8)
LR scheduler	Linear warmup with 0 warmup steps
Gradient clipping	Max norm = 1.0
BERT weights	Frozen (<code>torch.no_grad()</code>)
Device	CUDA if available, else CPU
Saved checkpoint	<code>bert_spoiler.pt</code>

5.1 Training Loop

Each epoch consists of a training phase and a validation phase. Gradient clipping is applied before each optimizer step to prevent exploding gradients, which is especially important when training on top of large pre-trained models:

```
for epoch in range(1, EPOCHS + 1):
    # Training
    model.train()
    for batch in train_loader:
        loss, _ = model(ids, mask, rel, lng, labels=lbl)
        loss.backward()
        clip_grad_norm_(model.parameters(), 1.0)
        optimizer.step(); scheduler.step()

    # Validation
    model.eval()
    with torch.no_grad():
        for batch in val_loader:
            loss, _ = model(ids, mask, rel, lng, labels=lbl)
```

6. BERT Tokenization & DataLoaders

6.1 Tokenizer Settings

Setting	Value	Reason
Model	bert-base-uncased	Matches the BERT encoder used in the classifier
Max length	128 tokens	Covers most review sentences; longer ones are truncated
Padding	max_length	Uniform tensor shapes for batching
Truncation	True	Silently truncates tokens beyond max_length
Return type	PyTorch tensors	Direct DataLoader compatibility
Token type IDs	Not returned	Single-sentence input; segment IDs unused

6.2 DataLoader Construction

Each DataLoader packages five tensors per sample: `input_ids`, `attention_mask`, `rel_pos`, `len_log`, and `labels`. The training loader uses `RandomSampler` for shuffling; validation and test loaders are sequential:

```
dataset = TensorDataset(
    encoded['input_ids'],      # [N, 128]
    encoded['attention_mask'], # [N, 128]
    rel_pos,                  # [N]
    len_log,                   # [N]
    labels                     # [N]
)
```

7. Evaluation

7.1 Metrics

The `evaluate()` function computes and visualizes the following metrics on both validation and test splits:

- Accuracy — overall fraction of correctly classified sentences.
- Precision, Recall, F1-score — computed per class (Spoiler / Not Spoiler) and as macro/weighted averages using `sklearn.classification_report`.
- Confusion Matrix — heatmap visualizing true vs. predicted label counts.
- ROC Curve & AUC — plots the trade-off between true positive rate and false positive rate across all classification thresholds.

7.2 Decision Threshold

The sigmoid output is thresholded at 0.5 to produce binary predictions:

```
probs = torch.sigmoid(logits).cpu().view(-1).tolist()
preds = [1 if p > 0.5 else 0 for p in probs]
```

Threshold Tuning

The default threshold of 0.5 may not be optimal for imbalanced classes. Consider tuning the threshold on the validation set by maximizing F1-score for the spoiler class, especially if recall on spoilers is prioritized over precision.

8. File Structure & Dependencies

8.1 Project Files

File	Description
spoiler_detection_simple.ipynb	Main notebook containing the full pipeline
goodreads_reviews_spoiler.json	Input dataset (JSONL format, one review per line)
bert_spoiler.pt	Saved model state dict after training

8.2 Dependencies

```
pip install transformers torch scikit-learn nltk pandas numpy matplotlib seaborn
tqdm
```

Library	Version Requirement	Purpose
torch	>=1.9	Model training, GPU acceleration, tensor ops
transformers	>=4.0	BERT model & tokenizer (HuggingFace)
scikit-learn	>=0.24	Train/val/test split, metrics (accuracy, F1, ROC)
nltk	>=3.6	Tokenization, stopword removal, lemmatization
pandas	>=1.3	DataFrame manipulation
numpy	>=1.20	Numerical operations
matplotlib	>=3.4	Plotting training curves
seaborn	>=0.11	Confusion matrix heatmap
tqdm	>=4.60	Progress bars for loops

8.3 NLTK Downloads Required

```

nltk.download('punkt')      # Sentence/word tokenizer
nltk.download('wordnet')    # Lemmatizer lexical database
nltk.download('stopwords')  # English stopwords list
nltk.download('punkt_tab')  # Updated punkt tokenizer tables

```

9. End-to-End Pipeline Summary

Step	Section	Input	Output
1. Install & Import	§1	—	Environment ready
2. Load Data	§2	goodreads_reviews_spoiler.json	df_raw (200k rows)
3. Expand to sentences	§3	df_raw	df with is_spoiler, sentence, rel_pos, len_log
4. Preprocess text	§4	df.sentence	df.sentence_clean
5. Clean & sample	§5	df	df (199k clean rows)
6. Split data	§6	df	X_train/val/test, y_train/val/test
7. Tokenize & batch	§7	X_train/val/test	train/val/test DataLoaders
8. Build model	§8	—	BertSpoilerClassifier instance
9. Train	§9	train_loader, val_loader	bert_spoiler.pt
10. Evaluate	§10	val/test_loader	Metrics, confusion matrix, ROC
11. Plot curves	§11	history	Training loss chart

10. Limitations & Potential Improvements

10.1 Current Limitations

- Frozen BERT — the encoder is not fine-tuned, which limits the model's ability to adapt BERT representations to the spoiler detection task. Fine-tuning the top BERT layers could improve performance.
- Preprocessing before BERT — removing stopwords and lemmatizing before BERT tokenization discards context that BERT's attention mechanism could leverage.
- Fixed threshold — the 0.5 sigmoid threshold is not optimized for the class distribution.

- No cross-book context — the model classifies sentences in isolation, without knowing the genre or specific book being reviewed.

10.2 Suggested Improvements

- Unfreeze the top 2–4 BERT layers for task-specific fine-tuning.
- Pass raw (un-preprocessed) sentences to the BERT tokenizer to preserve full linguistic context.
- Tune the classification threshold using the validation set F1-score.
- Experiment with longer max_length (256 or 512) for reviews with longer sentences.
- Add book genre as an additional feature — spoilers in a mystery novel differ contextually from those in a romance.
- Try DistilBERT or RoBERTa as the encoder for speed vs. accuracy trade-off exploration.

Appendix: Glossary

Term	Definition
BERT	Bidirectional Encoder Representations from Transformers. A pre-trained language model from Google.
CLS token	Special [CLS] token prepended to every BERT input. Its final hidden state is used as the sentence embedding.
Focal Loss	A modified cross-entropy loss that reduces the weight of easy examples, addressing class imbalance.
Lemmatization	Reducing a word to its base dictionary form (e.g., 'running' → 'run').
ROC / AUC	Receiver Operating Characteristic curve and Area Under Curve. Measures classifier performance across thresholds.
AdamW	Adam optimizer with decoupled weight decay. Standard for fine-tuning transformer models.
DataLoader	PyTorch utility that batches, shuffles, and serves tensors during training.
Stratified split	Train/val/test split that preserves the class distribution of the original dataset in each subset.